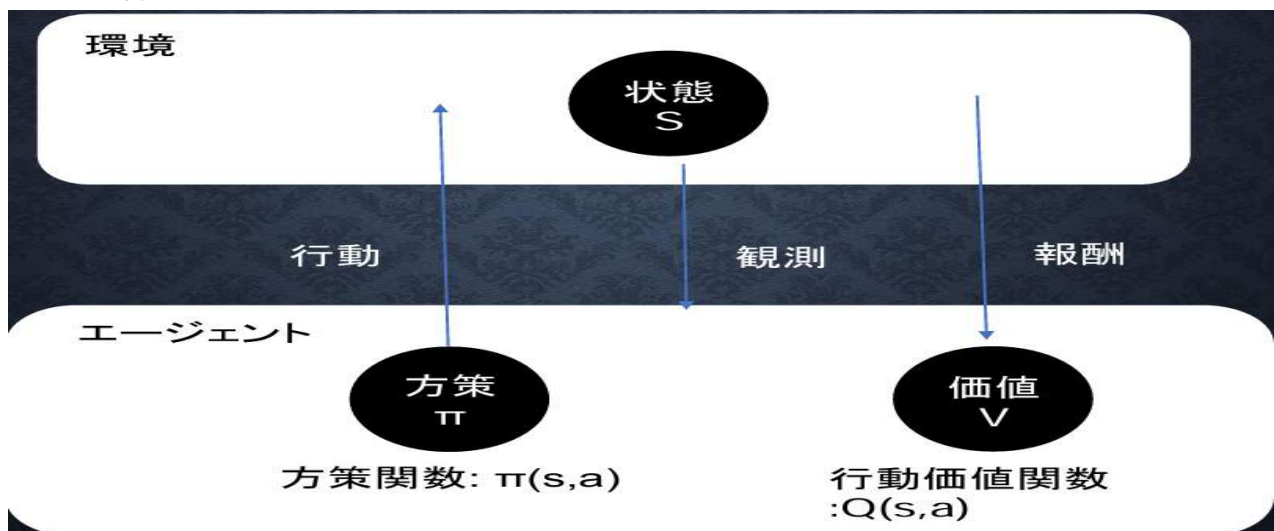


第一章 強化学習

強化学習・・・長期的に報酬を最大化できるように環境のなかで行動を選択
できるエージェントを作ること为目标とする機械学習の一分野
→行動の結果として与えられる利益(報酬)をもとに、
行動を決定する原理を改善していく仕組み

・強化学習のイメージ



・探索と利用のトレードオフ

環境について事前に完璧な知識があれば、最適な行動を予測し決定することは可能。
→どのような顧客にキャンペーンメールを送信すると、どのような行動を行うのかが
既知である状況。

→強化学習の場合、上記仮定は成り立たないとする。不完全な知識を元に行動しながら
データを収集。最適な行動を見つけていく。

→過去のデータで、ベストとされる行動のみを常に取り続けければ他にもっとベスト
な行動を見つけることはできない。

→検索が足りない状態⇔

↑ トレードオフの関係性

→未知の行動のみを常に取り続けければ、過去の経験が活かせない

→利用が足りない状態

・強化学習の差分

・強化学習と通常の教師あり、教師なし学習との違い

結論：目標が違う

- ・教師なし、あり学習では、データに含まれるパターンを見つけ出す
およびそのデータから予測することが目標
- ・強化学習では、優れた方策を見つけることが目標

- ・強化学習の歴史
 - ・冬の時代があったが、計算速度の進展により大規模な状態をもつ場合の強化学習を可能としつつある。
 - ・関数近似法と、Q学習を組み合わせる手法の登場
 - ・Q学習・・・行動価値関数を、行動する毎に更新することにより学習を進める方法
 - ・関数近似法・・・価値関数や方策関数を関数近似する手法
- ・価値関数・・・価値を表す関数としては、状態価値関数と行動価値関数の2種類
 - ある状態の価値に注目する場合は、状態価値関数状態と価値を組み合わせた価値に注目する場合は、行動価値関数
- ・方策関数・・・方策ベースの強化学習手法においてある状態でどのような行動をとるのか行動を採るのかの確率を与える関数
- ・関数の関係：エージェントは方策に基づいて行動する。
 - $\pi(s,a)$ ：VやQをもとにいう行動をとるか。
- ・方策反復法・・・方策をモデル化して最適化する手法→方策勾配法

$$\theta^{(t+1)} = \theta^{(t)} + \epsilon \nabla J(\theta)$$

↑

Jとは：方策の良さ

→定義しなければならない

- ・方策勾配法

定義方法：平均報酬・割引報酬和

→上記の定義に対応して、行動価値関数:Q(s,a)の定義を行い
方策勾配定理が成り立つ。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[(\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a))]$$

- ・TD (Temporal Difference Learning) 学習の概要
- ・TD学習・・・、動的計画法とモンテカルロ法の良いとこどりをした手法
- ・動的計画法
 - 利点：環境に対する知識を全て使えるため、最適な方策や価値関数を求めることができる
 - 欠点：環境に対する知識が事前に分かっている前提であるため、未知の環境での適用が難しく、限定的な課題でしか使用できない。また、一般的に計算量が多い。

- ・モンテカルロ法・・・シミュレーションや数値計算を乱数を用いて行う手法
利点：エピソードのサンプルをもとに学習可能で、環境に対する知識が不要。
欠点：エピソードが完了するまで価値関数や方策の更新ができない。また、変動が大きく学習が不安定になる場合がある。

- ・TD学習

- エピソードを進めながら各ステップで価値推定を更新 → エピソードが終了するのを待つ必要がなく高速、環境に対する知識も不要
- 各ステップの報酬と次の状態の価値推定を使用して学習するため、モンテカルロ法よりも学習が安定

※TD学習に限らず強化学習は、どの時点で新しい行動を試すべきか、既知の最良の行動を選択すべきかという「探索と利用のジレンマ」という問題を抱える。これらを解決するために、 ϵ -greedy方策やSoftmax方策など様々な方法が提案される。

	動的計画法	モンテカルロ法	TD学習
環境に対する知識	必要	不要	不要
主な更新対象	状態価値関数	行動価値関数	状態価値関数
ブートストラップ	使える	使えない	使える

- ・TD (Temporal Difference Learning) 学習の詳細

- ・TD学習の流れ

- ① エージェントは状態 s で方策 π に基づいて行動 a を選択
- ② 環境は報酬 r と次の状態 s' を返す
- ③ 状態 s の予測価値 $V(s)$ と実際の報酬 $r + \gamma V(s')$ の差分を時差誤差 δ として計算
→ $V(s)$ を δ で更新
 - ・時差誤差は実際の報酬と予測報酬のギャップであるため、誤差を小さくする方向に価値関数を調整する
 - ・実際には、時差誤差に学習率をかけることで、学習の安定性と収束時間のバランスを調整する

- ・TD学習の応用

TD学習の考え方を行動価値関数に拡張した様々なアルゴリズムが提案される

Q学習：最適な行動価値関数を直接学習するアルゴリズム。方策オフ型の手法で学習時の行動選択と実際の価値の更新が独立。

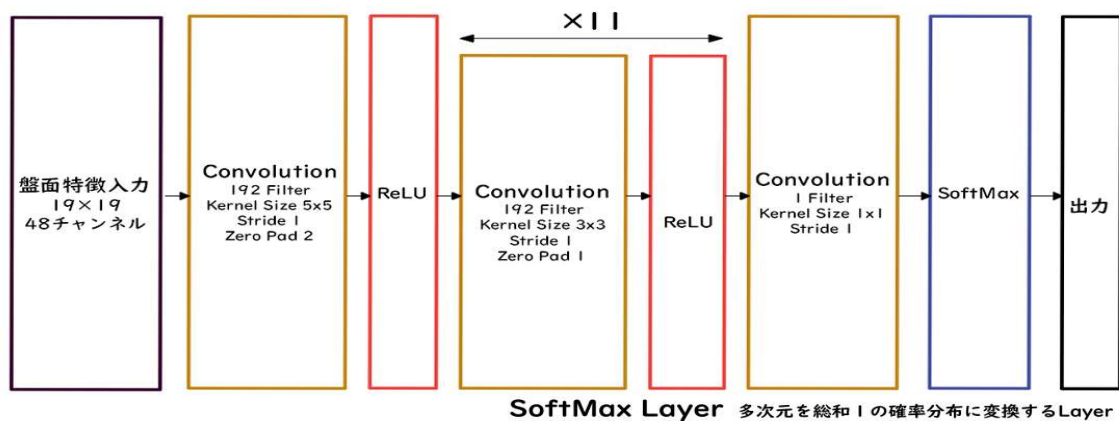
- 探索中の方策とは独立に最適な方策を学習できる特徴を持つ

SARSA：方策オン型の行動価値関数学習アルゴリズム。学習中の方策の影響を受けるため探索中の方策と実際に学習される価値が連動。

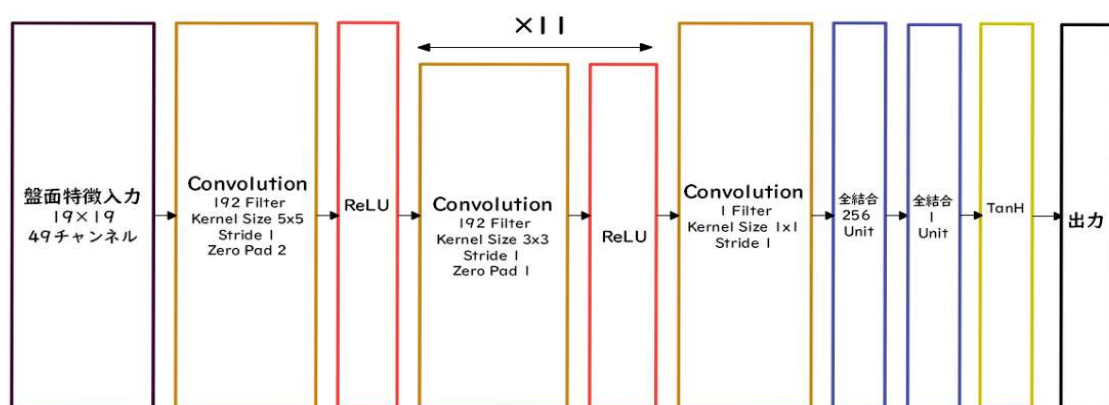
- 危険な状況や大きなペナルティをもたらす行動を選択するリスクが低くなる

第二章 Alpha Go

・ 方策関数



・ 価値観関数



RollOutPolicy・・・NNではなく線形の方策関数探索中に高速に着手確率を出すために使用される
→精度を落として、スピード重視

・ Alpha Go の学習

- 1、教師あり学習によるRollOutPolicyとPolicyNetの学習
- 2、強化学習によるPolicyNetの学習
- 3、強化学習によるValueNetの学習

・ PolicyNetの教師あり学習

KGS Go Server（ネット囲碁対局サイト）の棋譜データから3000万局面分の教師を用意し、教師と同じ着手を予測できるよう学習を行った。

具体的には、教師が着手した手を1とし残りを0とした19×19次元の配列を教師とし、それを分類問題として学習した。

この学習で作成したPolicyNetは57%ほどの精度である。

- ・ PolicyNetの強化学習

現状のPolicyNetとPolicyPoolからランダムに選択されたPolicyNetと対局シミュレーションを行い、その結果を用いて方策勾配法で学習を行った。PolicyPoolとは、PolicyNetの強化学習の過程を500Iterationごとに記録し保存しておいたものである。

現状のPolicyNet同士の対局ではなく、PolicyPoolに保存されているものとの対局を使用する理由は、対局に幅を持たせて過学習を防ごうというのが主
→この学習をminibatch size 128で1万回行った。

- ・ ValueNetの学習

PolicyNetを使用して対局シミュレーションを行い、その結果の勝敗を教師として学習した。

- ・ 教師データ作成の手順

- 1、まずSL PolicyNet(教師あり学習で作成したPolicyNet)でN手まで打つ。
- 2、N+1手目の手をランダムに選択し、その手で進めた局面をS (N+1) とする。
- 3、S (N+1) からRL PolicyNet (強化学習で作成したPolicyNet) で終局まで打ち、その勝敗報酬をRとする。

- ・ S(N+1) とRを教師データ対とし、損失関数を平均二乗誤差とし、回帰問題として学習
この学習をminibatch size 32で5000万回行った
→N手までとN+1手からのPolicyNetを別々にしてある理由は、過学習を防ぐため

- ・ モンテカルロ木探索・・・価値関数を取得するときの手法

コンピュータ囲碁ソフトでは現在もっとも有効とされている探索法。

- ・ 他のボードゲームではminimax探索やその派生形の $\alpha\beta$ 探索を使うことが多いが、盤面の価値や勝率予想値が必要となる
- ・ そこで、盤面評価値に頼らず末端評価値、つまり勝敗のみを使って探索を行うことができないかという発想で生まれた検索手法
- ・ 末端局面までPlayOutと呼ばれるランダムシミュレーションを多数回行い、その勝敗を集計して着手の優劣を決定
- ・ また、該当手のシミュレーション回数が一定数を超えたら、その手を着手したあとの局面をシミュレーション開始局面とするよう、探索木を成長
- ・ モンテカルロ木探索はこの木の成長を行うことによって、一定条件下において探索結果は最善手を返すということが理論的に証明さ

- ・ Alpha Go (Lee) のモンテカルロ木探索

Alpha Goのモンテカルロ木探索は選択、評価、バックアップ、成長という4つのステップで構成

- ・ AlphaGo Zero

- 1、教師あり学習を一切行わず、強化学習のみで作成
- 2、特徴入力からヒューリスティックな要素を排除し、石の配置のみにした
- 3、PolicyNetとValueNetを1つのネットワークに統合した
- 4、Residual Netを導入した
- 5、モンテカルロ木探索からRollOutシミュレーションをなくした

- ・ Residual Network・・・ネットワークにショートカット構造を追加して、勾配の爆発消失を抑える効果を狙ったもの

- ・ Residual NetworkResidual Networkを使うことにより、100層を超えるネットワークでの安定した学習が可能

- ・ 基本構造 (Residual Block)

Convolution→BatchNorm→ReLU→Convolution→BatchNorm→Add→ReLUの

Blockを1単位にして積み重ねる形

- ・ Residual Networkを使うことにより層数の違うNetworkのアンサンブル効果が得られているという説がある。

Alpha_Goの工夫

- ・ Residual Blockの工夫・・・Bottleneck、PreActivation
- ・ Network構造の工夫・・・WideResNet ・ PyramidNet

第三章 軽量化・高速化技術

- ・分散深層学習
 - ・分散深層学習 深層学習は多くのデータを使用したり、パラメータ調整のために多くの時間を使用したりするため、高速な計算が求められる。
 - ・複数の計算資源(ワーカー)を使用し、並列的にニューラルネットを構成することで、効率の良い学習を行いたい。
 - ・データ並列化、モデル並列化、GPUによる高速技術は不可欠である。
-
- ・データ並列化
 - ・データ並列化 親モデルを各ワーカーに子モデルとしてコピー
 - ・データを分割し、各ワーカーごとに計算させる
-
- ・データ並列化: 同期型
 - ・データ並列化: 同期型 データ並列化は各モデルのパラメータの合わせ方で、同期型か非同期型か決まる。
 - ・同期型・・・各ワーカーが計算が終わるのを待ち全ワーカーの勾配が出たところで勾配の平均を計算し、親モデルのパラメータを更新する。
-
- ・データ並列化: 非同期型
 - ・各ワーカーはお互いの計算を待たず、各子モデルごとに更新を行う。
学習が終わった子モデルはパラメータサーバにPushされる。
新たに学習を始める時は、パラメータサーバからPopしたモデルに対して学習していく。
-
- ・同期型と非同期型の比較
 - ・同期型と非同期型の比較処理のスピードは、お互いのワーカーの計算を待たない非同期型の方が早い。
 - ・非同期型は最新のモデルのパラメータを利用できないので、学習が不安定になりやすい。
 - ・現在は同期型の方が精度が良いことが多いので、主流となっている。
-
- ・モデル並列化
 - ・モデル並列化 親モデルを各ワーカーに分割し、それぞれのモデルを学習させる。全てのデータで学習が終わった後で、一つのモデルに復元。
 - ・モデルが大きい時はモデル並列化を、データが大きい時はデータ並列化をすると良い。
-
- ・モデル並列
 - ・モデル並列 モデルのパラメータ数が多いほど、スピードアップの効率も向上する。

- ・ 参照論文
- ・ 「Large Scale Distributed Deep Networks : 「ニューラルネットワークをとにかく巨大にし、データも計算資源も桁違いに投入したら何が起きるか」を実験的に示した論文
主張：「深層学習の性能は、アルゴリズム改良だけでなく、計算資源とスケールによって質的に変わる」
- ・ GPUによる高速化
- ・ GPUによる高速化GPGPU (General-purpose on GPU)
→ 元々の使用目的であるグラフィック以外の用途で使われるGPUの総称
- ・ CPU
→ 高性能なコアが少数
→ 複雑で連続的な処理が得意
- ・ GPU
→ 比較的低性能なコアが多数
→ 簡単な並列処理が得意
→ ニューラルネットの学習は単純な行列演算が多いので、高速化が可能
- ・ GPGPU開発環境
- ・ CUDA
→ GPU上で並列コンピューティングを行うためのプラットフォーム
→ NVIDIA社が開発しているGPUのみで使用可能。
→ Deep Learning用に提供されているので、使いやすい
- ・ OpenCL
→ オープンな並列コンピューティングのプラットフォーム
→ NVIDIA社以外の会社(Intel, AMD, ARMなど)のGPUからでも使用可能。
→ Deep Learning用の計算に特化しているわけではない。
- ・ Deep Learningフレームワーク(Tensorflow, Pytorch)内で実装されているので、使用する際は指定すれば良い
- ・ モデルの軽量化・・・モデルの精度を維持しつつパラメータや演算回数を低減する手法の総称
- ・ 通常は 低演算性能での利用が必要とされるIoTなどエッジコンピューティングでの利用

- ・モデルの軽量化の利用

モデルの軽量化はモバイル、IoT 機器、エッジコンピューティングの利用において有用な手法

→モバイル端末や IoT はパソコンに比べ性能が大きく劣る

→モデルの軽量化は計算の高速化と省メモリ化を行うためモバイル、IoT 機器と相性が良い手法になる。

- ・軽量化の手法・・・量子化 ・蒸留 ・プルーニング

- ・量子化(Quantization)

通常のパラメータの 64 bit 浮動小数点を 32 bit など下位の精度に落

とすことでメモリと演算処理の削減を行う

- ・メリット：計算の高速化、小メモリ化

- ・デメリット：精度の低下

- ・計算の高速化

倍精度演算(64 bit)と単精度演算(32 bit)は演算性能が大きく違うため、量子化により精度を落とすことによりより多くの計算をすることができる。

- ・省メモリ化

ニューロンの重みを浮動小数点のbit数を少なくし有効桁数を下げることによってニューロンのメモリサイズを小さくすることができ、多くのメモリを消費するモデルのメモリ使用量を抑えることができる。

- ・精度の低下

ニューロンが表現できる少数の有効桁が小さくなるとモデルの表現力が低下する。

→学習した結果ニューロンが重みを表現できなくなる。

→ただし、実際の問題では倍精度を単精度にしてもほぼ精度は変わらない。

- ・極端な量子化

→誤差の大きな式になる。

→量子化する際は極端に精度が落ちない程度に量子化をしなければならない。

- ・蒸留

精度の高いモデルはニューロンの規模が大きなモデルになっている。

そのため、推論に多くのメモリと演算処理が必要。

→規模の大きなモデルの知識を使い軽量なモデルの作成を行う。

- ・モデルの簡約化

学習済みの精度の高いモデルの知識を軽量なモデルへ継承させる

- ・教師モデルと生徒モデル

蒸留は教師モデルと生徒モデルの2つで構成される

教師モデル・・・予測精度の高い、複雑なモデルやアンサンブルされたモデル

生徒モデル・・・教師モデルをもとに作られる軽量なモデル

- ・教師モデルと生徒モデル

教師モデルの重みを固定し生徒モデルの重みを更新していく。

誤差は教師モデルと生徒モデルのそれぞれの誤差を使い重みを更新していく。

- ・蒸留の利点

蒸留によって少ない学習回数でより精度の良いモデルを作成することができる。

- ・プルーニング

モデルの精度に寄与が少ないニューロンを削減することでモデルの軽量化

高速化が見込まれる。

- ・計算の高速化

寄与の少ないニューロンの削減を行いモデルの圧縮を行うことで高速化に計算できる。

- ・ニューロンの削減

ニューロンの削減の手法は重みが閾値以下の場合ニューロンを削減し再学習を行う

- ・ニューロン数と精度

閾値を高くするとニューロンは大きく削減できるが精度も減少する。

- ・モデルの軽量化まとめ

- ・量子化：重みの精度を下げるにより計算の高速化と省メモリ化を行う技術

- ・蒸留：複雑で精度の良い教師モデルから軽量の生徒モデルを効率よく学習を行う技術

- ・プルーニング：寄与の少ないニューロンをモデルから削減し高速化と省メモリ化を行う技術

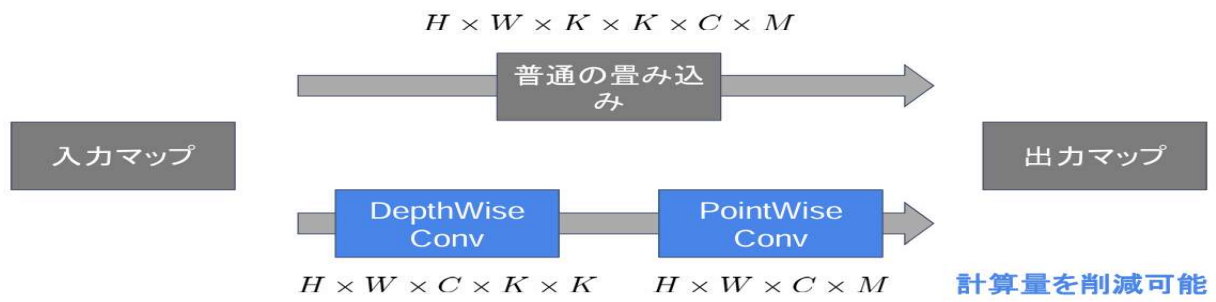
第四章 応用技術

- MobileNet
- 論文タイトル：MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications
- 提案手法
 - ディープラーニングモデルは精度は良いが、その分ネットワークが深くなり計算量が増える。
 - 計算量が増えると、多くの計算リソースが必要で、お金がかかってしまう。
 - ディープラーニングモデルの軽量化・高速化・高精度化を実現（その名の通りモバイルなネットワーク）
- 一般的な畳み込みレイヤー
 - 入力特徴マップ（チャンネル数）： $H \times W \times C$
 - 畳み込みカーネルのサイズ： $K \times K \times C$
 - 出力チャンネル数（フィルタ数）： M
- 一般的な畳み込みレイヤーは計算量が多い
 - MobileNetsはDepthwise Convolution とPointwise Convolution 組み合わせで軽量化を実現
- Depthwise Convolution
- 仕組み
 - 入力マップのチャンネルごとに畳み込みを実施
 - 出力マップをそれらと結合（入力マップのチャンネル数と同じになる）
 - 通常の畳み込みカーネルは全ての層にかかっていることを考えると計算量が大幅に削減可能
 - 各層ごとの畳み込みなので層間の関係性は全く考慮されない。通常はPW畳み込みとセットで使うことで解決
- Pointwise Convolution
- 仕組み
 - 1×1 convとも呼ばれる（正確には $1 \times 1 \times c$ ）
 - 入力マップのポイントごとに畳み込みを実施
 - 出力マップ(チャンネル数)はフィルタ数分だけ作成可能

- MobileNetの全体像

MobileNet

Depth-wise separable convolutions



第五章 ResNet（転移学習）

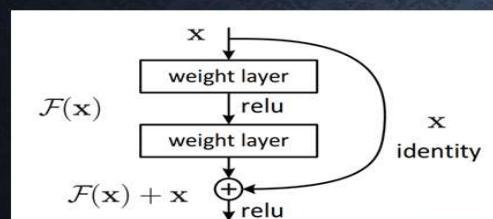
- ・画像識別モデルの実利用
- ・画像識別モデルの実利用効率的な学習方法
教師あり学習において、目的とするタスクでの教師データが少ない場合に別の目的で学習した学習済みモデルを再利用する転移学習
- ・異なるドメインの学習結果を利用する
→ImageNetによる事前学習
- ・ImageNetは1400万件以上の写真のデータセット様々なAI/MLモデルの評価基準になっており、学習済みモデルも多く公開

ImageNetを1000分類で分類した教師データを利用。ResNetにより学習。以下はサンプル。

No.	Index	Label
1	0	tench, Tinca tinca
2	1	goldfish, Carassius auratus
3	2	great white shark, white shark, man-eater, man-eating shark, Carcharodon carcharias
4	3	tiger shark, Galeocerdo cuvieri
5	4	hammerhead, hammerhead shark

- ・ResNet・・・「残差ブロック（Residual Block）」と「スキップ接続（Skip Connection）」を導入することで、従来のモデルでは難しかった「非常に深い層」を持つニューラルネットワークの学習を可能にした画期的な画像認識モデル
- ・ResNet: SkipConnection

中間層部分



- 深い層の積み重ねでも学習可能に
 - 勾配消失の回避
 - 勾配爆発の回避
- 中間層の部分出力: $H(x)$
- 残差ブロック: $H(x) = F(x) + x$
- 学習部分: $F(x)$

- ・ResNet: Bottleneck構造
同一計算コストで1層多い構造
途中の層で3x3の畳込みを行う

- WideResnet: 構造

ResNetにおけるフィルタ数をK倍

→ 畳込みチャンネル数が増加

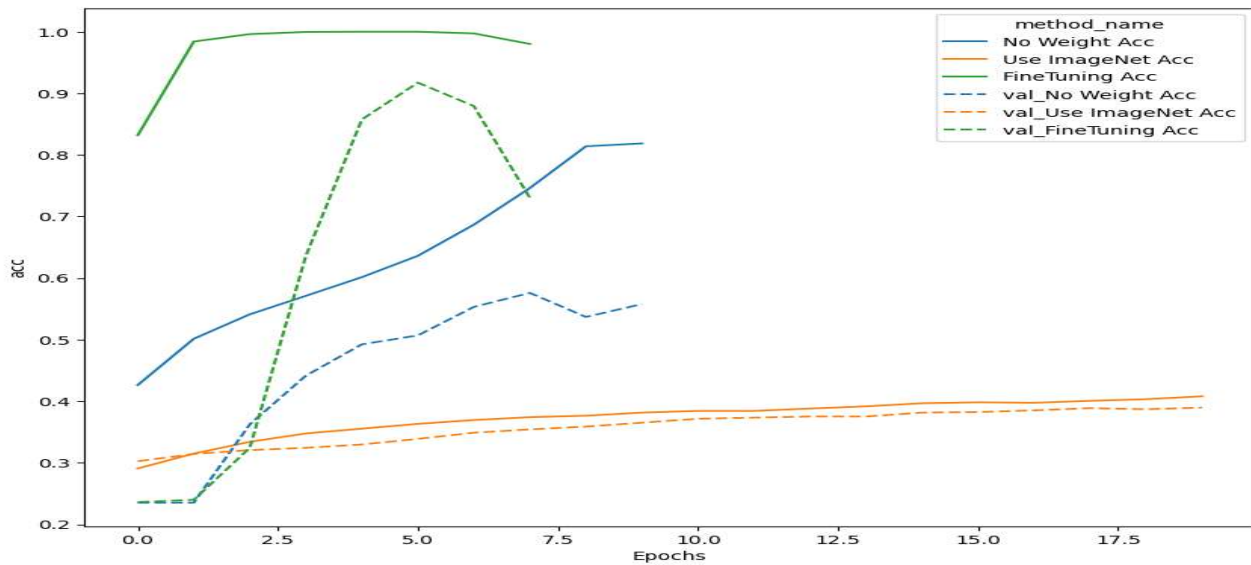
→ 高速・高精度の学習が可能に

→ GPUの特性に合った動作

ResNetに比べ層数を浅くした

DropoutをResidualブロックに導入

<実践演習：4_1_transfer-learning.ipynb>



<実践演習：4_2_wide_resnet.ipynb>

kerasとHunの不整合によりエラー発生

第六章 EfficientNet

- ・ AlexNet以降は、CNNモデルを大規模にスケールアップすることで精度を改善するアプローチが主流となった

- ・ 課題：精度を向上できたものの、モデルが複雑で高コスト

→2019年に開発されたEfficientNet

→開発当時の最高水準の精度を上回り、同時にパラメータ数を大幅に減少

要約」

ConvNetsにおけるネットワークの深さや広さ、解像度などがモデルの性能にどう影響を

及ぼすかを調べ、Compound Coefficient(複合係数) というものを導入することで性能を上げた。これにより作ったEfficientNet-B7はImageNetでSoTAである84.4% top-1 Acc., 97.1% top-5 Acc. を叩き出した。しかも、モデル自体は今までのSoTAモデルと比べて8.4倍も小さく6.1倍も速い。転移学習でもいい結果を残している。

この論文ではモデルの大きさに応じてEfficientNet-B0からEfficientNet-B7まである。

引用：Qita

<実践演習：EfficientNetを使った画像分類（転移学習の基本形）>

- ・ ImageNetで事前学習済みモデルを読み込み
- ・ 自前のクラス分類に使う

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model

# 事前学習済みEfficientNetB0を読み込む
base_model = EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

# ベースモデルは学習させない（凍結）
base_model.trainable = False

# 独自の分類層を追加
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation="relu")(x)
outputs = Dense(10, activation="softmax")(x) # クラス数=10の例

model = Model(inputs=base_model.input, outputs=outputs)

# モデルのコンパイル
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

第七章 Vision Transformer

Vision Transformer (ViT)は、言語処理に用いられるTransformerを用いた、画像分類タスク用のモデル

- Vision Transformerのデータ表現と入力

画像をパッチに分割し、パッチごとの特徴量を系列データとしてTransformerに入力する。パッチごとの特徴量は、ピクセル値をFlatten処理・Embedding処理(埋め込み)を行いPositionEmbedding情報を加えたもの。

入力系列の1番目に、分類タスク用に特別な”[CLS]”トークンを連結するして入力する。従って、入力系列の長さは、パッチ数+ 1となる。

- Vision Transformerのアーキテクチャ

言語処理におけるTransformerのエンコーダーとほぼ同様の構造である。

エンコーダー部分からの出力における1トークン目の特徴量を、MLPHeadに入力、最終的な分類結果を出力する。

モデルサイズの違いにより、Base/Large/Hugeの3つが存在する。

- Vision Transformerにおける事前学習(Pre-training)とファインチューニング(Fine-tuning)

ViTの事前学習は、教師ラベル付きの巨大データセットで行われ、性能がデータセット規模の影響を受ける

ファインチューニングでは、MLPHead部分を取り換えることで、分類タスクにおけるクラス数の違いに対応する。

ファインチューニングで、事前学習より高い解像度の画像に、Position Embeddingの変更のみで対応できる。

- 性能とその評価

事前学習におけるデータセットが大規模な場合において、既存手法より高性能。

事前学習のデータセットが小規模な場合、既存手法(CNN)に比べ低性能。

同計算量において、既存手法より高性能。大計算量域でさらなる性能向上の余地がある。

<実践演習：Vision Transformer の最小実用例)

[5]
✓ 2 秒

```
import torch
import timm
from PIL import Image
from torchvision import transforms

# デバイス設定
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ViTモデルをロード (事前学習済み)
model = timm.create_model(
    "vit_base_patch16_224",
    pretrained=True,
    num_classes=1000
)
model.to(device)
model.eval()

# 画像前処理
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.5, 0.5, 0.5),
        std=(0.5, 0.5, 0.5)
    )
])

# 画像読み込み
image = Image.open("/content/sample.jpg").convert("RGB")
x = transform(image).unsqueeze(0).to(device)

# 推論
with torch.no_grad():
    outputs = model(x)
    predicted_class = outputs.argmax(dim=1)

print("予測クラスID:", predicted_class.item())
```

▼

... 予測クラスID: 549